

FastAPI 中间件实践

两种注册方式

装饰器（适合简单、一次性逻辑）

```
from fastapi import FastAPI, Request, Response
import time

app = FastAPI()

@app.middleware("http")
async def add_process_time_header(request: Request, call_next) -> Response:
    start_time = time.perf_counter()
    response = await call_next(request)
    process_time = time.perf_counter() - start_time
    response.headers["X-Process-Time"] = str(process_time)
    return response
```

`app.add_middleware()`（适合可复用/第三方/需要配置参数的中间件）

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:8080"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

装饰器本质是 `add_middleware` 的语法糖，但无法传配置参数。

中间件函数签名

```
from typing import Awaitable, Callable

async def my_middleware(
    request: Request,
    call_next: Callable[[Request], Awaitable[Response]]
) -> Response:
    # 请求前：可以读取/修改 request
    response = await call_next(request)
```

```
# 响应后：可以读取/修改 response
return response
```

- `call_next(request)` 把控制权交给下一层中间件或路由
- 必须返回 `Response`，否则请求会挂起
- 短路场景（如认证失败）：直接返回 `Response` 而不调用 `call_next`

执行顺序（洋葱模型）

```
app.add_middleware(MiddlewareA) # 第二个注册
app.add_middleware(MiddlewareB) # 第一个注册 → 但它是最外层
```

请求进入 → MiddlewareB → MiddlewareA → 路由处理 → MiddlewareA → MiddlewareB → 响应返回

后注册的中间件包裹在最外层：

- 认证中间件应该后注册（最先拦截请求）
- CORS 中间件通常放最外层（最后一个 `add_middleware`）

常见内置中间件

中间件	用途
CORSMiddleware	跨域资源共享
GZipMiddleware	响应压缩
TrustedHostMiddleware	限制允许的 Host 头
HTTPSRedirectMiddleware	强制 HTTPS

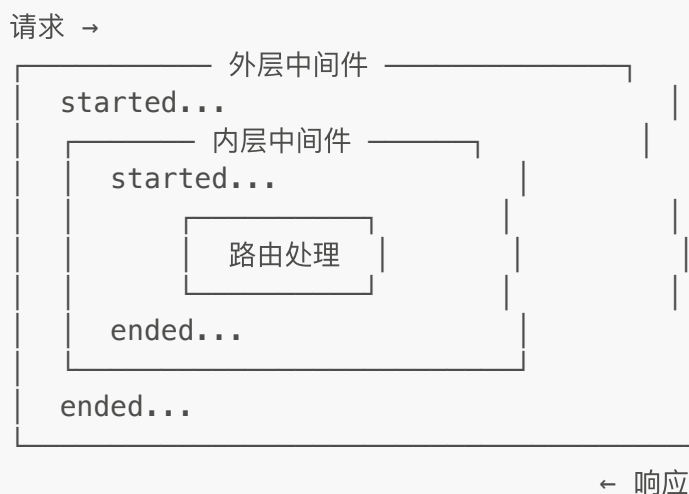
推荐注册顺序（从内到外）

```
app.add_middleware(GZipMiddleware, minimum_size=500) # 最内层
app.add_middleware(AuthMiddleware) # 认证
app.add_middleware( # 最外层
    CORSMiddleware,
    allow_origins=["..."],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

洋葱模型详解

`call_next` 是洋葱的分界点——之前的代码处理请求（往里走），之后的代码处理响应（往外走）：

```
async def my_middleware(request, call_next):  
    # ← 请求阶段：洋葱往里走  
    logger.info("started")  
  
    response = await call_next(request) # ← 穿到下一层  
  
    # ← 响应阶段：洋葱往外走  
    logger.info("ended")  
    return response
```



进的顺序和出的顺序是反的。注册顺序与执行顺序也是反的——后注册的在外层先执行，本质是装饰器堆叠：

```
# @app.middleware 按代码顺序注册 A、B  
# 等价于：  
app = B(A(原始app))  
#      ^ 外层，请求先到 B
```

这个模型在很多框架里本质相同：Express (`next()`)、Koa (`await next()`)、Django (`get_response`)。

纯 ASGI 类中间件

基本结构

```
from starlette.types import ASGIApp, Receive, Scope, Send  
  
class MyMiddleware:  
    def __init__(self, app: ASGIApp):  
        self.app = app
```

```
async def __call__(self, scope: Scope, receive: Receive, send: Send):
    # 非 http 请求必须直接透传 (lifespan、websocket 等)
    if scope["type"] != "http":
        await self.app(scope, receive, send)
        return

    # 请求处理逻辑...

    await self.app(scope, receive, send) # 等价于 call_next
```

为什么必须过滤 scope type

ASGI 有三种 scope type:

type	含义
"http"	普通 HTTP 请求
"websocket"	WebSocket 连接
"lifespan"	应用启动/关闭生命周期事件

`@app.middleware("http")` 装饰器自动过滤只处理 http 类型。纯 ASGI 类中间件的 `__call__` 会被所有类型调用，应用启动时第一个进来的就是 `lifespan`，不过滤会导致错误。

手动构造请求相关对象

在纯 ASGI 中间件里没有现成的 `Request` 对象，需要从 `scope` 手动构造：

```
from starlette.requests import Request
from starlette.datastructures import Headers, QueryParams

# 完整 Request (需要读 body 时传 receive)
request = Request(scope, receive)

# 只需要查询参数 (更轻量)
query_params = QueryParams(scope.get("query_string", b""))

# 只需要请求头
headers = Headers(scope=scope)
```

手动读取 body

没有 `request.body()` 封装，需要直接调用 `receive`：

```
# 简单读取
message = await receive()
body = message.get("body", b"")
```

```
# 大 body 可能分块传输, 需循环
body = b""
while True:
    message = await receive()
    body += message.get("body", b"")
    if not message.get("more_body", False):
        break
```

`receive` 是一次性消费的——读完后下游拿不到数据。必须重建 `receive` 给下游:

```
async def new_receive():
    return {"type": "http.request", "body": body}

await self.app(scope, new_receive, send)
```

注意事项

- 不要在中间件里读 `request.body()`: `body` 是流式的, 读一次就消耗了, 后续路由拿不到。如果确实需要, 必须重新构造 `receive`
- 中间件与路由传递数据: 用 `request.state` (如 `request.state.user = current_user`), 不要用全局变量
- 避免阻塞 I/O: 中间件里做阻塞操作会拖慢整个事件循环, 必要时用 `asyncio.to_thread()` 包裹
- FastAPI 中间件底层完全基于 **Starlette**, 所有 Starlette 中间件可直接使用
- 无意义的 `try/except`: `try: ... except SomeError as e: raise e` 捕获后原封不动重抛, 和不写 `try` 完全等价。只在需要记录日志、转换异常类型、返回默认值等场景才用 `try/except`